

Kuaishou Recommender System

By Sami Lodi



Kuaishou / Kuai



Kuaishou

- Large Chinese short-video & Livestreaming Platform.
- User swipe through videos rapidly and interact with.
- Generates massive behavioural data from millions of daily users.
- Users watching Ads helps generate revenue for stakeholders to continue improving App



Domain Characteristics and Aim

- **Short-Video platform** = Fast, swipe-based consumption
- Preferences can Change Quickly -> high-volume, **high-frequency interactions**.
- Aim to **recommend possible next videos for user to watch**

Notable Observations

Highly observed User-Item Interaction Matrix:

- 99.6% Density
- Everyone has interacted with multiple videos.
- Especially good since it enables fair, stable, reproducible evaluations.

Contains Mostly Implicit and User Behavioural Data:

- Not rich in content data
- Item_daily_features.csv contains data mostly about how all user's interacted with video rather than specific users
- Not much Data on each specific User-Item Interaction

s p a r s e

	7					6
	7	6	3		4	
	4	3				
4	2					
				3	2	4

© Matt Eding

DENSE

0	7	0	0	0	0	6
0	7	6	3	0	4	0
0	4	3	0	0	0	0
4	2	0	0	0	0	0
0	0	0	0	3	2	4

Collaborative Filtering (CF)

Collaborative Filtering (CF):

- Predicts a user's interest based on patterns of behaviour across users and items [2].

Why CF for KuaiRec?

- Stable Item Space
- Item driven user behaviour
- Ideal for implicit Feedback (watch_ratio)
- Industry Standard

[2, 3, 4, 5]



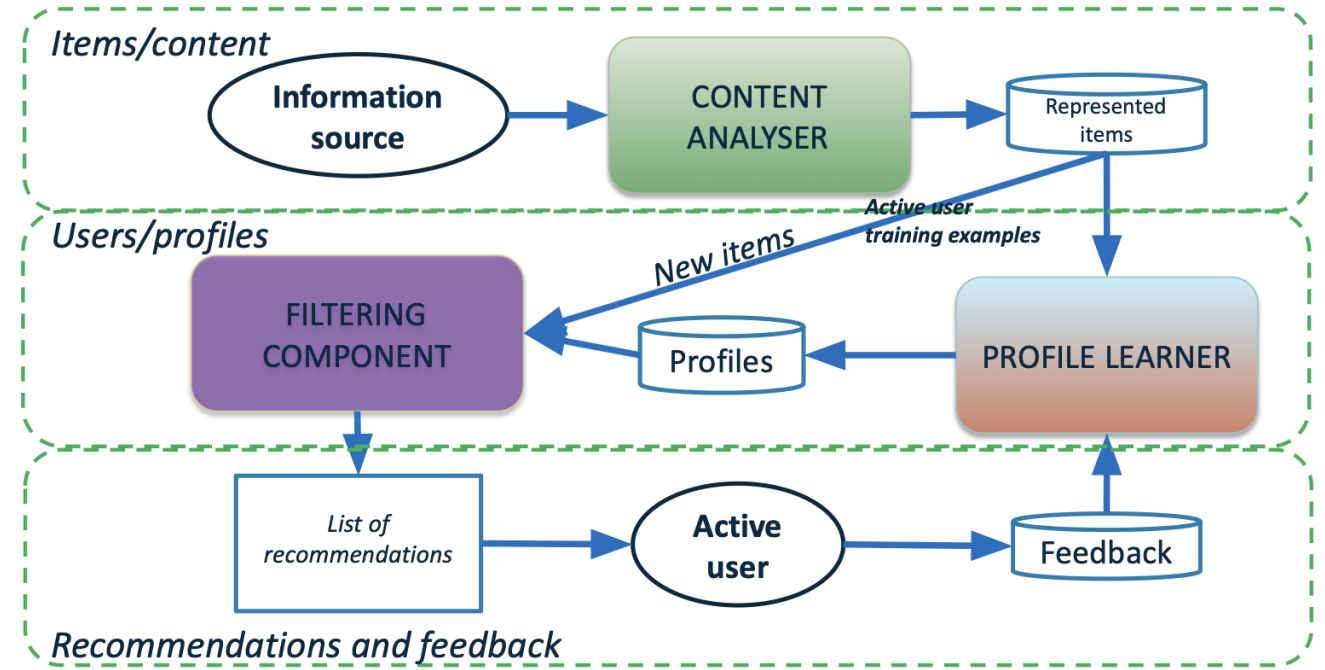
Content-Based Filtering (CBF)

Content-Based Filtering (CBF):

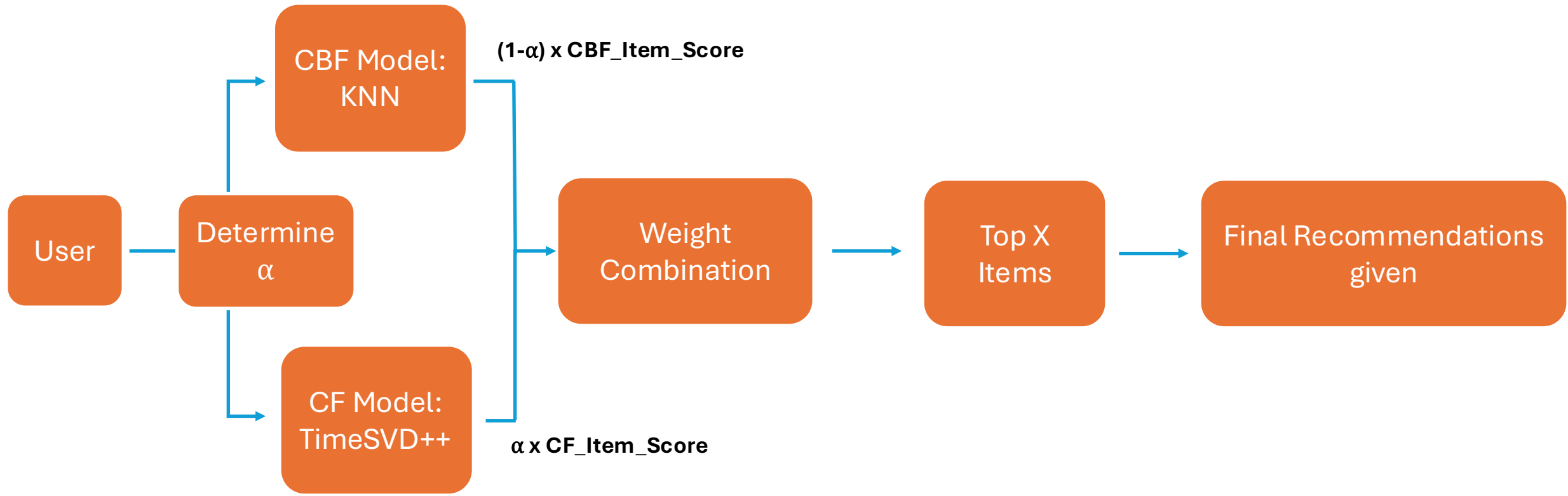
- Predict a user's interest by matching item features to the features of items the user has previously engaged with.

Why CBF for KuaiRec?

- Recommendations can be easily explained
- Data is present which describe video content (From multiple CSV files)
- Reduce Cold-Start Problem



RS1 Architecture: Contextually-Aware Dynamically-Weighted Hybrid Model



RS1: Data Preparation - General

Merged KuaiRec source tables:

- (*small_matrix, social_network, user_features, item_daily_features, item_categories, caption_category*)
- **Why:** To combine interaction data with user and item metadata.

Sampled a 100k interaction subset:

- **Why:** To ensure computational feasibility while preserving data diversity.

Removed extremely sparse users and items:

- **Why:** To reduce noise and improve model stability.

Performed a time-based train-test split (80:20):

- **Why:** To prevent information leakage from future interactions.

Separated data into collaborative and content-based DataFrames:

- **Why:** To support a hybrid recommender system with distinct CF and CBF pipelines.

```
# 1. Merge User Features
merged_df = pd.merge(small_matrix, user_features, on='user_id', how='left')

# 2. Merge Social Network
merged_df = pd.merge(merged_df, social_network, on='user_id', how='left')

# 3. Merge Item Categories
merged_df = pd.merge(merged_df, item_categories, on='video_id', how='left')

# 4. Merge Caption Category
merged_df = pd.merge(merged_df, caption_category, on='video_id', how='left')

# 5. Merge Item Daily Features
# Joining on both video_id and date to get the features for that specific day
merged_df = pd.merge(merged_df, item_daily_features, on=['video_id', 'date'], how='left')

print(f"Merged dataset shape: {merged_df.shape}")
```

Merged dataset shape: (4494578, 105)

```
# SPLIT merged_df INTO collab_df + content_df USING SCHEMAS

def normalize_col(col: str) -> str:
    # Strip common merge suffixes for matching
    return re.sub(r"_(x|y|_left|_right|_social|_user|_itemcat|_caption|_daily)$", "", col)

normalized_map = {c: normalize_col(c) for c in merged_df.columns}

collab_keep = [c for c, nc in normalized_map.items() if nc in COLLAB_COLS]
content_keep = [c for c, nc in normalized_map.items() if nc in CONTENT_COLS]

collab_df = merged_df[collab_keep].copy()
content_df = merged_df[content_keep].copy()

# Normalise column names in derived DFs
collab_df.columns = [normalized_map[c] for c in collab_keep]
content_df.columns = [normalized_map[c] for c in content_keep]

# Remove duplicate columns caused by normalisation
collab_df = collab_df.loc[:, ~collab_df.columns.duplicated(keep="first")]
content_df = content_df.loc[:, ~content_df.columns.duplicated(keep="first")]

# Debug overlap/unassigned
overlap = sorted(set(collab_df.columns) & set(content_df.columns))
unassigned = sorted([normalized_map[c] for c in merged_df.columns] - (COLLAB_COLS | CONTENT_COLS))

print("Overlap columns count:", len(overlap))
if overlap:
    print("Overlap sample:", overlap[:30])

print("Unassigned normalized columns count:", len(unassigned))
if unassigned:
    print("Unassigned sample:", unassigned[:30])
```

```
# Drop rows where date is NaN (approx 4% of data, also missing time)
print(f"Original small_matrix shape: {small_matrix.shape}")
small_matrix = small_matrix.dropna(subset=['date'])
print(f"Shape after dropping null dates: {small_matrix.shape}")

# Convert date in small_matrix to int to match item_daily_features
# small_matrix['date'] looks like 20200705.0 (float)
small_matrix['date'] = small_matrix['date'].astype(int)

# Ensure item_daily_features date is int
item_daily_features['date'] = item_daily_features['date'].astype(int)

print("Date columns converted to integer.")
```

```
sample_size = 100000
if len(merged_df) > sample_size:
    sampled_df = merged_df.sample(n=sample_size, random_state=42)
    output_sample_file = 'KuaiRec 2-2.0/data/merged_kuairc_dataset_100k.csv'
    sampled_df.to_csv(output_sample_file, index=False)
    print(f"Sampled dataset ({sample_size} rows) saved to {output_sample_file}")
else:
    print(f"Dataset is smaller than {sample_size} rows, skipping sampling.")
```

Sampled dataset (100000 rows) saved to KuaiRec 2-2.0/data/merged_kuairc_dataset_100k.csv

RS1: Data Preparation - TimeSVD++

Defined a binary interaction signal from engagement behaviour:

- rating = 1 for sufficiently engaged interactions; non-interactions are unobserved
- **Why:** KuaiRec provides implicit feedback only; TimeSVD++ can be applied by binarising interactions while leveraging its implicit-feedback term (SVD++ component)

Extracted temporal information from timestamps:

- Discretised time into bins (e.g. morning, afternoon, evening, night)
- **Why:** Enables TimeSVD++ to model temporal drift in user preferences and item popularity.

Constructed a clean user-item interaction matrix:

- One row per user-item interaction with time and implicit rating
- **Why:** Provides the required input structure for matrix factorisation-based collaborative filtering.

```
# collab_df: one row per interaction
if not {"user_id", "video_id"}.issubset(collab_df.columns):
    raise ValueError("collab_df must contain user_id and video_id after the split!")

if "timestamp" in collab_df.columns:
    collab_df["timestamp"] = pd.to_numeric(collab_df["timestamp"], errors="coerce")
    before = collab_df.shape[0]
    collab_df = collab_df.drop_duplicates(subset=["user_id", "video_id", "timestamp"])
    after = collab_df.shape[0]
    print(f"collab_df deduped on ['user_id', 'video_id', 'timestamp']: {before} -> {after}")
else:
    subset = [c for c in ["user_id", "video_id", "date", "time"] if c in collab_df.columns]
    before = collab_df.shape[0]
    collab_df = collab_df.drop_duplicates(subset=subset)
    after = collab_df.shape[0]
    print(f"collab_df deduped on {subset}: {before} -> {after}")

# rating from watch_ratio >= 2
if "watch_ratio" in collab_df.columns and "rating" not in collab_df.columns:
    collab_df["rating"] = (collab_df["watch_ratio"] >= 2).astype(int)
```

```
# TimeSVD++ Feature Engineering
# Create Time Bins (0-3) from 'time'
if "time" in collab_df.columns:
    collab_df["time_dt"] = pd.to_datetime(collab_df["time"], errors="coerce")
    collab_df["hour"] = collab_df["time_dt"].dt.hour

    collab_df["time_bin"] = pd.cut(
        collab_df["hour"],
        bins=[-1, 5, 11, 17, 23],
        labels=[0, 1, 2, 3] # Night, Morning, Afternoon, Evening
    ).astype("Int64")

# Create Day Index (0, 1, 2...) from 'date'
if "date" in collab_df.columns:
    if not pd.api.types.is_datetime64_any_dtype(collab_df["date"]):
        collab_df["date"] = pd.to_datetime(collab_df["date"].astype(str), format="%Y%m%d", errors="coerce")

    min_date = collab_df["date"].min()
    collab_df["day_index"] = (collab_df["date"] - min_date).dt.days

# 3. DROP old time columns (Cleanup)
cols_to_drop = ["time", "timestamp", "date", "hour", "time_dt", "tod_bucket"]
print(f"Dropping original time columns to save space: {cols_to_drop}")
collab_df.drop(columns=[c for c in cols_to_drop if c in collab_df.columns], inplace=True, errors="ignore")

# content_df: one row per video_id (latest date if present)
if "video_id" not in content_df.columns:
    raise ValueError("content_df must contain video_id after split.")

if "date" in content_df.columns:
    # date is YYYYMMDD -> parse safely
    if not pd.api.types.is_datetime64_any_dtype(content_df["date"]):
        content_df["date"] = pd.to_datetime(content_df["date"].astype(str), format="%Y%m%d", errors="coerce")

    content_df = content_df.sort_values("date").groupby("video_id", as_index=False).tail(1)
    # Optional: Drop date from content_df too if you don't need it anymore
    # content_df.drop(columns=["date"], inplace=True)
else:
    content_df = content_df.drop_duplicates(subset=["video_id"])

assert content_df["video_id"].is_unique, "content_df must have unique video_id (one row per item)."
```

```
# rating from watch_ratio >= 2
if "watch_ratio" in collab_df.columns and "rating" not in collab_df.columns:
    collab_df["rating"] = (collab_df["watch_ratio"] >= 2).astype(int)
```

RS1: Data Preparation for KNN

Cleaned and tokenised Chinese text fields:

- Removed emojis [16] and applied *jieba* tokenisation
- Why:** Enables meaningful vectorisation of captions and tags in Chinese.

Identified positively engaged items using watch ratio:

- Used a watch ratio threshold to represent user interest
- Why:** Provides a behavioural signal for which items to base similarity recommendations on.

Encoded structured item features:

- One-hot encoded categorical metadata
- Why:** Allows non-text features to contribute to item similarity.

Vectorised textual content using TF-IDF:

- Applied TF-IDF to captions and tags
- Why:** Captures semantic similarity between videos in high-dimensional sparse space. [7]

Learned dense embeddings for author and music IDs:

- Used Word2Vec to embed author and music identifiers
- Why:** Captures latent stylistic similarity beyond explicit metadata.

=== Learning Author & Music Embeddings ===

[A] Building User Sequences
content_df has author_id: True
content_df has music_id: True
Merged rows: 100000
author_id non-null: 100000
music_id non-null: 100000
Author sequences: 1411 users
Music sequences: 1411 users
Unique authors: 2031, appearing >=2 times: 2031
Unique music: 2689, appearing >=2 times: 2689

[B] Training Word2Vec Models
✓ Author vocab: 2031 authors
✓ Music vocab: 2689 tracks

[C] Generating Embeddings for content_df
Author embeddings: (3327, 64)
Music embeddings: (3327, 32)
Coverage: 3327/3327 videos with author emb
Coverage: 3327/3327 videos with music emb

✓ Embeddings ready!

```
# Basic Chinese stopwords
CHINESE_STOPWORDS = {
    '的', '了', '在', '是', '我', '有', '和', '就', '不', '人', '都', '一',
    '一个', '上', '也', '很', '到', '说', '要', '去', '你', '会', '着', '没有',
    '着', '好', '自己', '这', '中', '么', '个', '来', '为', '能', '他', '对',
    '她', '用', '与', '可以', '里', '但', '将', '之', '从', '而', '把', '被',
    '让', '无', '又', '及', '更', '或', '等', '已', '给', '并', '且', '还',
    '只', '最', '再', '当', '如', '于', '才', '所', '以', '吗', '呢', '吧'
}

# Emoji regex pattern
EMOJI_REGEX = re.compile(
    "["
    "\U0001F300-\U0001F5FF"
    "\U0001F600-\U0001F64F"
    "\U0001F680-\U0001F6FF"
    "\U0001F700-\U0001F77F"
    "\U0001F780-\U0001F7FF"
    "\U0001F800-\U0001F8FF"
    "\U0001F900-\U0001F9FF"
    "\U0001FA00-\U0001FAFF"
    "\U0001FB00-\U0001FBFF"
    "]"*,
    flags=re.UNICODE
)
```

```
def segment_chinese_text(text, remove_stopwords=True, keep_english=True):
    """
    Preprocess and segment Chinese text using jieba

    Steps:
    1. Handle edge cases (NaN, empty strings)
    2. Remove/keep special characters
    3. Clean up whitespace
    4. Segment using jieba
    5. Optionally remove stopwords

    Args:
        text: Input text string
        remove_stopwords: Whether to remove Chinese stopwords
        keep_english: Whether to keep English letters and numbers

    Returns:
        Segmented text string with words separated by spaces
    """
```

```
def preprocess_kualrec_text(content_df, text_columns=None, remove_emojis=True, remove_stopwords=True, keep_english=True):
    """
    Preprocess all KUALREC columns in KualRec dataset

    Args:
        df: DataFrame containing KualRec data
        text_columns: List of column names to process. If None, uses default columns
        remove_emojis: Whether to remove emojis first
        remove_stopwords: Whether to remove stopwords during segmentation
        keep_english: Whether to keep English text
        inplace: If True, replaces original columns. If False, creates new _segmented columns

    Returns:
        DataFrame with processed KUALREC columns
    """
    # Default KUALREC columns in KualRec
    if text_columns is None:
        text_columns = [
            'manual_cover_text',
            'caption',
            'topic_tag',
            'first_level_category_name',
            'second_level_category_name',
            'third_level_category_name'
        ]

    # Filter to only columns that exist in the dataframe
    text_columns = [col for col in text_columns if col in df.columns]

    print(f"Processing {len(text_columns)} KUALREC columns...")

    # Step 1: Remove emojis if requested
    if remove_emojis:
        print("Removing emojis...")
        for col in text_columns:
            df[col] = df[col].apply(remove_emojis)
        print("✓ Emojis removed")
```

```
# Step 2: Segment Chinese text
print("\nSegmenting Chinese text...")
for col in text_columns:
    # Process the text
    processed = df[col].apply(
        lambda x: segment_chinese_text(x, remove_stopwords, keep_english)
    )

    # Either replace original or create new column
    if inplace:
        df[col] = processed
        target_col = col
    else:
        target_col = f'{col}_segmented'
        df[target_col] = processed

    # Show progress
    non_empty = (df[target_col] != '').sum()
    print(f"✓ {col}: {non_empty}/{len(df)} non-empty")

print("\n Text preprocessing complete!")

return df
```

```
# TF-IDF + MULTI-MODAL FEATURES

from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.preprocessing import StandardScaler, MultiLabelBinarizer, OneHotEncoder, LabelEncoder
from sklearn.feature_selection import chi2, SelectKBest
from scipy.sparse import hstack, csr_matrix
import numpy as np
import ast

print("\n=== Building Content Features (Refactored) ===")
def safe_text(series):
    return series.fillna("").astype(str)

# ===== A: TEXT TF-IDF WITH CHI-SQUARE SELECTION =====
print("\n[A] Text Features (TF-IDF + Chi-Square Selection)")
# 1. Use SEGMENTED text (already processed by jieba in earlier cells)
content_df["text_combined_seg"] = (
    safe_text(content_df["caption"]) + " " +
    safe_text(content_df["manual_cover_text"])
).str.strip()

# 2. Frequency pruning with CountVectorizer
cv = CountVectorizer(min_df=10, max_df=0.5, token_pattern=r"(?u)\b\w+\b")
X_counts = cv.fit_transform(content_df["text_combined_seg"])
print(f"Initial vocab size: {len(cv.get_feature_names_out())}")
# 3. Chi-square feature selection using first_level_category as target
le = LabelEncoder()
y_category = le.fit_transform(content_df["first_level_category_name"].fillna("UNKNOWN"))
selector = SelectKBest(chi2, k=win1000, X_counts=X_counts.shape[1]) # Cap at vocab size
X_selected = selector.fit_transform(X_counts, y_category)
selected_vocab = cv.get_feature_names_out()[selector.get_support()]
print(f"After chi-square: {len(selected_vocab)} features")

# 4. TF-IDF with selected vocabulary
tfidf_text = TfidfVectorizer(vocabulary=selected_vocab, sublinear_tf=True)
text_matrix = tfidf_text.fit_transform(content_df["text_combined_seg"])
print(f"Text TF-IDF matrix: {text_matrix.shape}")
```

Memory-Based CBF: K Nearest Neighbour (KNN)

What is Memory-Based CF:

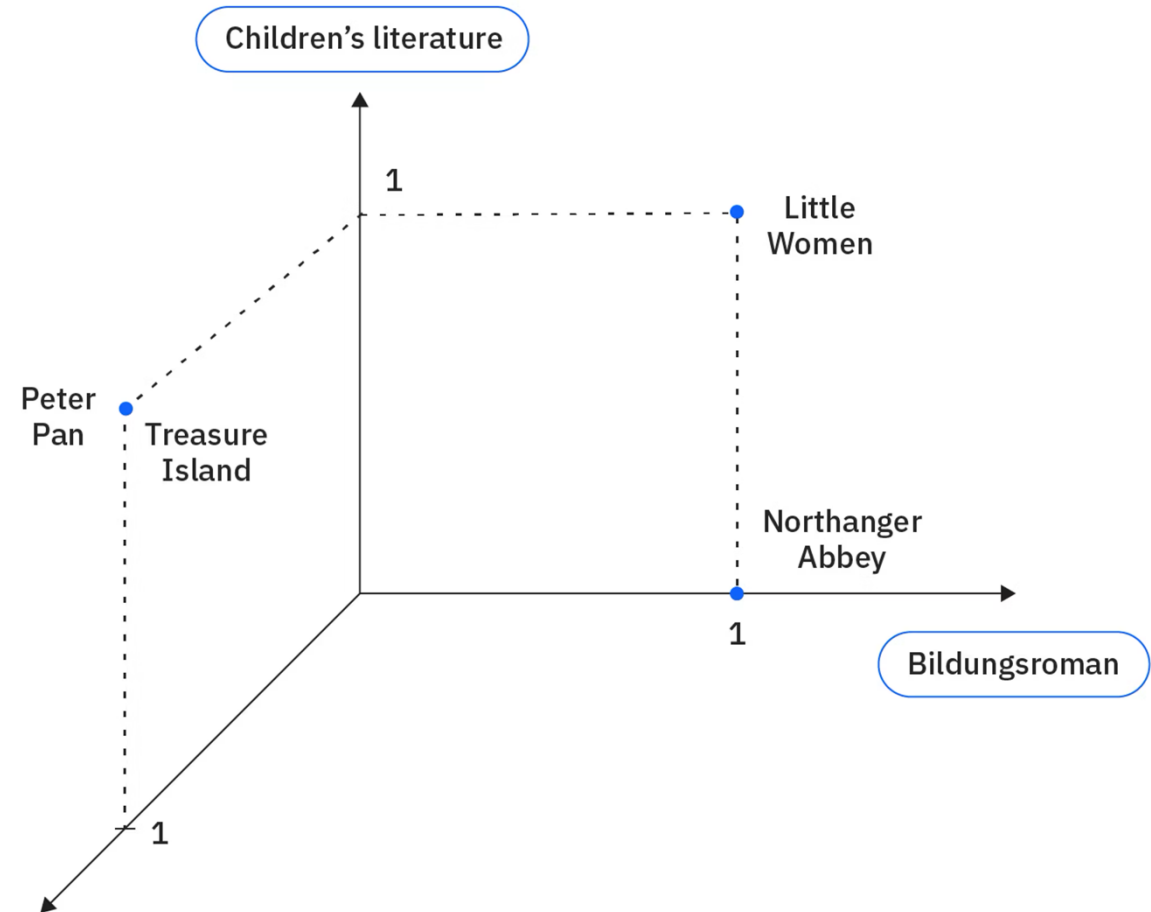
- Recommend videos similar to those user already interacted with.
- Similarity computed from item content feature vectors. [3]

Why KNN for KuaiRec:

- Directly models Item Similarity
- Good for High Dimensionality Sparse data
- Many features does not overlap
- Handles Cold-Start problem better than many CF options

How KNN work:

- Computes similarity between items using their content feature vectors.
- Recommends the most similar items to those the user has previously engaged with.



Memory-Based CBF: K Nearest Neighbour (KNN)

Items are represented as vectors in a high-dimensional content feature space (TF-IDF text features + categorical and embedding features).

$\text{sim}(i, j) = \cos$ of angle between Item Vectors.

The neighbourhood size K was selected empirically to balance locality and coverage.

Why Cosine Similarity:

- Robust to sparse, high-dimensional TF-IDF vectors
- Measures angular similarity (semantic alignment) rather than magnitude
- Standard measure for item-item CBF baselines

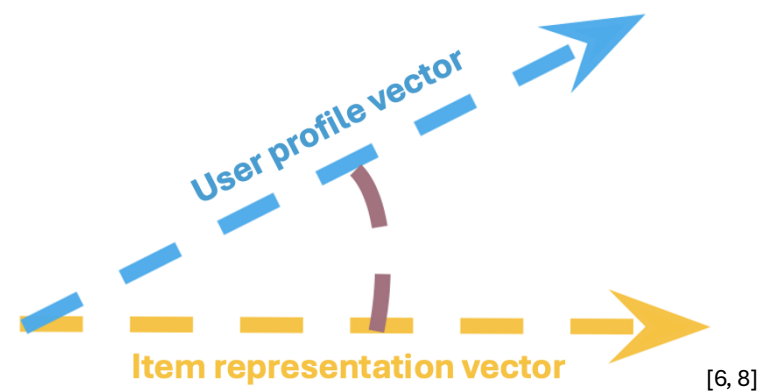
Neighbourhood selection

- Top-K most similar items per item

Scoring

- Items scored based on similarity to items the user previously engaged with (implicit positives)

$$\text{sim}(i, j) = \cos(\vec{i}, \vec{j}) = \frac{\vec{i} \cdot \vec{j}}{\|\vec{i}\|_2 * \|\vec{j}\|_2}$$



```
knn_model = NearestNeighbors(n_neighbors=200, metric='cosine', algorithm='brute', n_jobs=-1)
knn_model.fit(content_features)
print("KNN Model Indexed.")
```

Model-Based CF: TimeSVD++

What is Model-Based CF?

- Use ratings to learn a predictive model
- Transform items and users to the same latent factor space

Why TimeSVD++ for KuaiRec:

- KuaiRec provides **no explicit ratings**
- User preferences and item popularity are **time-dependent**

How TimeSVD++ works:

- **Learns latent user and item factors** from user-item interaction data using matrix factorisation.
- **Incorporates time dynamics**, allowing user preferences and item popularity to change over time.
- **Uses implicit feedback** from past interactions to refine user representations and improve predictions.

[4]

The diagram illustrates the TimeSVD++ model equation for predicting a rating $\hat{r}_{ui}$ based on various factors. The equation is:

$$\hat{r}_{ui} = \mu + b_i(t_{ui}) + b_u(t_{ui}) + q_i^T \left(p_u(t_{ui}) + |\mathbf{R}(u)|^{-\frac{1}{2}} \sum_{j \in \mathbf{R}(u)} y_j \right)$$

Annotations with arrows pointing to the equation components:

- Global average rating / interaction level**: Points to μ .
- Item bias that changes over time**: Points to $b_i(t_{ui})$.
- User bias that changes over time**: Points to $b_u(t_{ui})$.
- latent factor vector for item i** : Points to q_i^T .
- normalisation**: Points to $|\mathbf{R}(u)|^{-\frac{1}{2}}$.
- latent vector for item j as implicit feedback**: Points to y_j .
- Items the user has interacted with**: Points to the summation set $j \in \mathbf{R}(u)$.
- user's latent preference vector at time t** : Points to $p_u(t_{ui})$.

Model-Based CF: TimeSVD++

Performed a constrained grid search to balance model capacity, regularisation, and temporal stability:

- Latent dimensionality
- Learning rate
- Regularisation strength
- Number of epochs
- Implicit feedback weighting

Selection criterion

- Optimised on RMSE
- Aligned with top-K recommendation performance

```
# Prepare data for TimeSVD++
# Map users and items to contiguous indices
unique_users = train_df['user_id'].unique()
unique_items = train_df['video_id'].unique()

user_to_idx = {u: i for i, u in enumerate(unique_users)}
item_to_idx = {it: i for i, it in enumerate(unique_items)}
idx_to_item = {i: it for it, i in item_to_idx.items()}

train_user_indices = train_df['user_id'].map(user_to_idx).values
train_item_indices = train_df['video_id'].map(item_to_idx).values
train_ratings = train_df['rating'].values

# Use time_bin if available, otherwise use row index as proxy
if 'time_bin' in train_df.columns:
    train_timestamps = train_df['time_bin'].values
else:
    train_timestamps = None

# Train TimeSVD++
timesvd_model = TimeSVDpp(
    n_factors=30,
    n_epochs=20,
    lr=0.005,
    reg=0.02,
    n_time_bins=30,
    random_state=SEED
)

timesvd_model.fit(
    train_user_indices,
    train_item_indices,
    train_ratings,
    timestamps=train_timestamps
)

print(f'✅ TimeSVD++ Model Trained')
```

Training TimeSVD++ for 20 epochs...

```
Epoch 0: RMSE = 0.2214
Epoch 5: RMSE = 0.1978
Epoch 10: RMSE = 0.1926
Epoch 15: RMSE = 0.1895
```

```
# Parameter Grids

KNN_GRID = {
    'n_neighbors': list(range(50, 501, 50)),
    'metric': ['cosine'],
    'tfidf_max_features': [30000, 50000],
    'tfidf_ngrams': [(1, 1), (1, 2), (1, 3)],
    'selectkbest_k': [5000, 10000]
}

TIMESVD_GRID = {
    'n_factors': list(range(50, 201, 50)),
    'n_epochs': [100, 200],
    'lr': [0.005, 0.01],
    'reg': [0.01, 0.02, 0.05, 0.1]
}

SASREC_GRID = {
    'd_model': [64, 128],
    'n_heads': [2, 4],
    'n_blocks': [2, 3],
    'dropout': [0.2, 0.3],
    'lr': [0.0005, 0.001],
    'batch_size': [128, 256],
    'n_epochs': [100, 200]
}

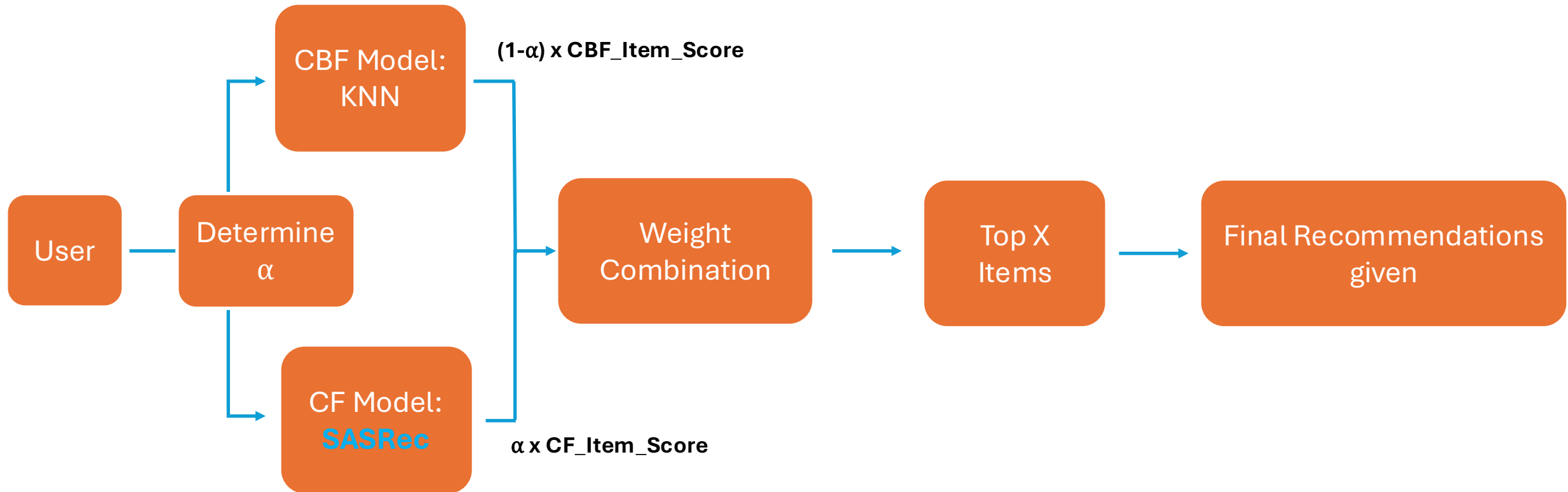
# Calculate totals
from functools import reduce
import operator

knn_total = reduce(operator.mul, [len(v) for v in KNN_GRID.values()])
timesvd_total = reduce(operator.mul, [len(v) for v in TIMESVD_GRID.values()])
sasrec_total = reduce(operator.mul, [len(v) for v in SASREC_GRID.values()])

print(f'KNN combinations: {knn_total}')
print(f'TimeSVD++ combinations: {timesvd_total}')
print(f'SASRec combinations: {sasrec_total}')
print(f'Total: {knn_total + timesvd_total + sasrec_total}')

if TEST_MODE:
    print(f'\n^ TEST MODE: Will test 20 configs per model')
```

RS2 Architecture: Contextually-Aware Dynamically-Weighted Hybrid Model



RS2: Data Preparation for SASRec

Select Relevant Columns:

- User ID, Item ID, and Timestamp (required for sequence ordering)
- All interactions treated as implicit positive feedback (ratings not used)

Sequence Construction:

- SASRec models sequential patterns ("watched A, then B, then C")
- Temporal order critical - random order destroys signal

Sequence Truncation/Padding:

- Median KuaiRec user history $\approx$ 68 interactions
- Maxlen = 79 captures majority (Upper Quartile) of sequences without truncation
- Longer histories yield diminishing returns for next-item prediction
- Balances temporal coverage and computational cost
- Sequences are left-padded with a padding token (0) and truncated from the front to preserve recency.

Why this matters:

- Preserves temporal order
- Allows SASRec to learn recency and transition patterns

```
# BUILD CONTIGUOUS ID MAPPINGS

print("\n[Step 7] Building contiguous ID mappings...")
print(" (0 is reserved for padding token)")

# Create sorted lists of unique IDs
sorted_users = sorted(sasrec_df['user_id'].unique())
sorted_items = sorted(sasrec_df['video_id'].unique())

# Create mappings (1..N, 0 reserved for padding)
user2idx = {uid: idx+1 for idx, uid in enumerate(sorted_users)}
item2idx = {iid: idx+1 for idx, iid in enumerate(sorted_items)}

# Reverse mappings
idx2user = {idx: uid for uid, idx in user2idx.items()}
idx2item = {idx: iid for iid, idx in item2idx.items()}

# Apply mappings
sasrec_df['uid'] = sasrec_df['user_id'].map(user2idx)
sasrec_df['iid'] = sasrec_df['video_id'].map(item2idx)

print(f" User mapping: original {min(sorted_users)}-{max(sorted_users)} -> compact 1-{len(user2idx)}")
print(f" Item mapping: original {min(sorted_items)}-{max(sorted_items)} -> compact 1-{len(item2idx)}")
print(f" Total unique users: {len(user2idx):,}")
print(f" Total unique items: {len(item2idx):,}")

# Verify no missing mappings
assert sasrec_df['uid'].notna().all(), "User mapping failed"
assert sasrec_df['iid'].notna().all(), "Item mapping failed"
print(" All IDs successfully mapped")
```

```
=== Sequence Length Statistics ===
Total users: 1411
Total items: 3327

Sequence Lengths:
Mean: 68.1
Median: 68.0
Min: 44
Max: 96
25th percentile: 62.0
75th percentile: 74.0

🔍 Data-Driven Maxlen:
90th percentile: 79
Using maxlen: 79

📊 9.4% of sequences will be truncated (maxlen=79)
✅ maxlen=79 seems reasonable
```

```
# DIAGNOSTIC: Sequence Statistics

seq_lengths = [len(seq) for seq in user_sequences.values()]

print(f" Total users: {len(user_sequences)}")
print(f" Total items: {len(item2idx)}")
print(f" Sequence Lengths:")
print(f" Mean: {np.mean(seq_lengths):.1f}")
print(f" Median: {np.median(seq_lengths):.1f}")
print(f" Min: {min(seq_lengths)}")
print(f" Max: {max(seq_lengths)}")
print(f" 25th percentile: {np.percentile(seq_lengths, 25):.1f}")
print(f" 75th percentile: {np.percentile(seq_lengths, 75):.1f}")

# Compute data-driven maxlen (90th percentile)
MAXLEN = int(np.percentile(seq_lengths, 90))
MAXLEN = min(MAXLEN, 200) # Cap at 200 for compute efficiency

print(f" Data-Driven Maxlen:")
print(f" 90th percentile: {int(np.percentile(seq_lengths, 90))}")
print(f" Using maxlen: (MAXLEN)")

# Check maxlen appropriateness
pct_truncated = sum(1 for l in seq_lengths if l > MAXLEN) / len(seq_lengths) * 100
print(f" {pct_truncated:.1f}% of sequences will be truncated (maxlen={MAXLEN})")

if pct_truncated > 30:
    print(f"Consider increasing maxlen (>{MAXLEN}) to capture longer sequences")
elif pct_truncated < 5:
    print(f"maxlen={MAXLEN} is appropriate for this data")
else:
    print(f"maxlen={MAXLEN} seems reasonable")
```

Advanced Model-Based Sequential CF: SASRec

What is Model-Based CF:

Learns a **parametric predictive model** from user-item interaction data.

Represents users via their **interaction sequences**, not static profiles.

Learns to predict future user interactions from past behaviour.

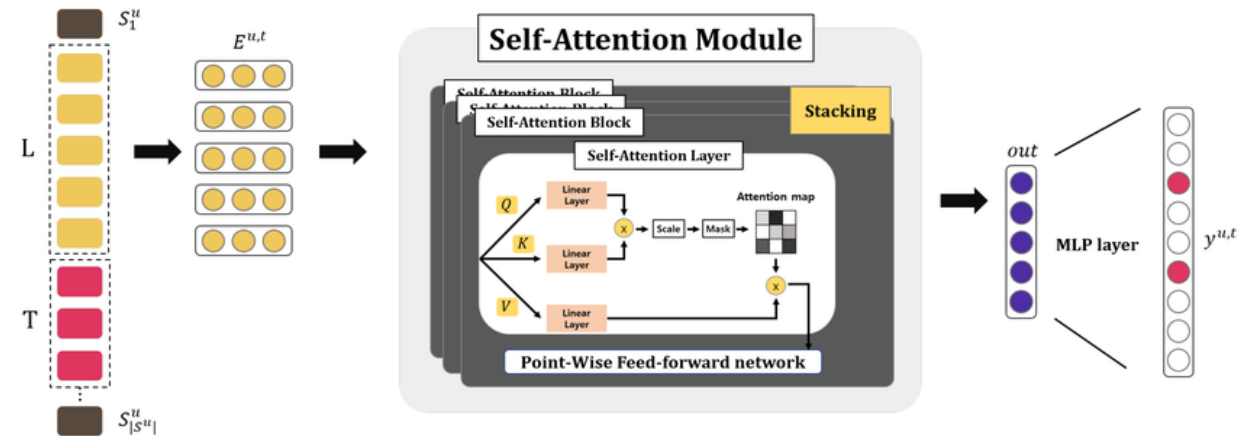
Why SASRec for KuaiRec:

- User behaviour is **order-dependent**, not just time-dependent.
- Captures **short-term intent** and **recency effects**.
- Avoids manual time discretisation.

How SASRec works:

- Represents each user as an **ordered sequence of interacted items**.
- Uses **Transformer self-attention** to model dependencies between past interactions.
- Predicts the **next item** given the past interactions.
- High-capacity non-linear model enabling complex sequential dependency modelling

[11]



[9]

Advanced Model-Based Sequential CF: SASRec

How SASRec more advanced than TimeSVD++:

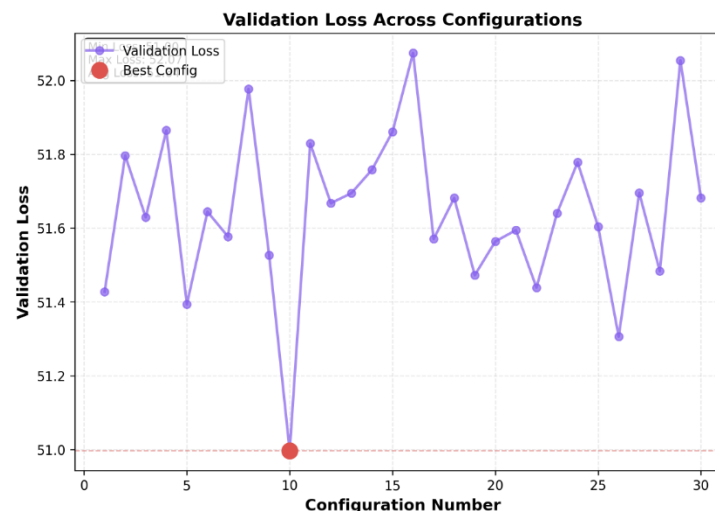
- Uses **causal (masked) self-attention** to model ordered interaction sequences and short-term user intent without future information leakage

Parameters tuned using Grid Search Strategy:

- **Latent dimension (d_model):** controls representation capacity
- **Number of self-attention blocks:** controls depth of sequential reasoning
- **Number of attention heads:** controls modelling of multiple behaviour patterns
- **Dropout rate:** regularisation for sparse user histories
- **Learning rate:** training stability and convergence speed
- **Number of epochs:** ensures convergence without overfitting

Selection criterion:

- **Binary cross-entropy loss with negative sampling**
- optimising next-item ranking under implicit feedback.



```
SASREC_GRID = {
    'd_model': [128],
    'n_heads': [2, 4],
    'n_blocks': [2, 3],
    'dropout': [0.2, 0.3],
    'lr': [0.0005, 0.001],
    'batch_size': [128, 256],
    'n_epochs': [200]
}
```

```
# Initialize SASRec
sasrec_model = SASRec(
    n_items=mappings['n_items'],
    maxlen=mappings['maxlen'],
    d_model=64,
    n_heads=2,
    n_blocks=2,
    dropout=0.2
).to(device)

# Optimizer with learning rate warmup
optimizer = torch.optim.Adam(sasrec_model.parameters(), lr=0.01, betas=(0.9, 0.98), eps=1e-8)

print('Training SASRec for 20 epochs...')
print('-' * 60)

# Training loop
for epoch in range(100):
    avg_loss = train_sasrec(sasrec_model, train_loader, optimizer, device, epoch)

    if epoch % 5 == 0 or epoch == 19:
        print(f'Epoch {epoch+1:2d}/100: Loss = {avg_loss:.4f}')

print('-' * 60)
print('SASRec training complete!')
```

```
# Calculate combinations
from functools import reduce
import operator
n_combos = reduce(operator.mul, [len(v) for v in SASREC_GRID.values()])
print(f"\n[2/3] Starting SASRec Grid Search ({n_combos} configurations)...")
print(f" Estimated time: ~{n_combos * 9 / 60:.1f} hours")

# Run grid search
sasrec_results, sasrec_best, sasrec_score = grid_search('sasrec', SASREC_GRID, val_data, max_combinations=None)

all_results = {
    'sasrec': {
        'results': sasrec_results,
        'best_params': sasrec_best,
        'best_score': sasrec_score
    }
}
```

Context-Aware Hybrid using Dynamic Weights

Mechanism (what the model does):

- RS1 combines TimeSVD++ (CF) and KNN (CBF) using a weighted score:
- Final Score = $(\alpha \times \text{CF_Score}_i) + ((1 - \alpha) \times \text{CBF_Score}_i)$
- The weight α is determined by user activity level. ($\alpha \in [0, 1]$)

How α is set:

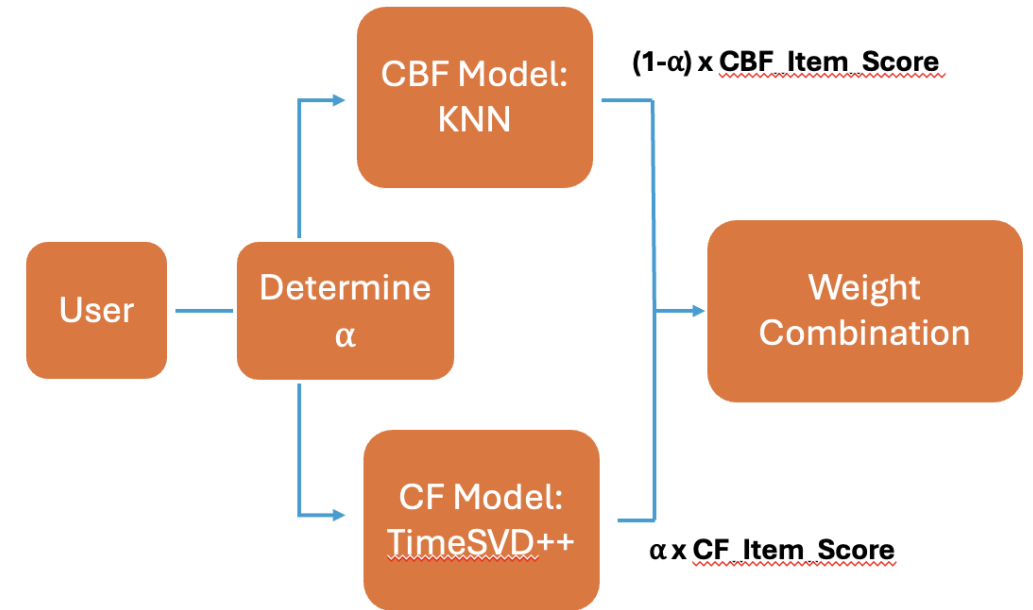
- User activity level is derived from an explicit KuaiRec user activity feature based on interaction distribution across content categories.
- High-activity users $\rightarrow$ larger $\alpha \rightarrow$ rely more on collaborative filtering
- Low-activity / new users $\rightarrow$ smaller $\alpha \rightarrow$ rely more on content-based filtering

Why dynamic weighting is used:

- Active users have richer interaction histories, making CF more reliable
- New or sparse users benefit more from content similarity
- This reduces cold-start effects while enabling progressively stronger personalisation as user interaction data increases.

Role in RS1:

- Provides an adaptive yet interpretable hybrid baseline
- Allows fair comparison with RS2, which replaces heuristics with learned models



```
def get_alpha(user_id):
    if user_id not in user_meta_lookup.index:
        return 0.5
    row = user_meta_lookup.loc[user_id]
    if row['is_lowactive_period'] == 1:
        return 0.2 # Trust content
    deg = str(row['user_active_degree'])
    if deg == 'full_active': return 0.8
    if deg == 'high_active': return 0.7
    if deg == 'middle_active': return 0.6
    return 0.5
```

Contextual Modelling using Time

RS1: Explicit Temporal Context

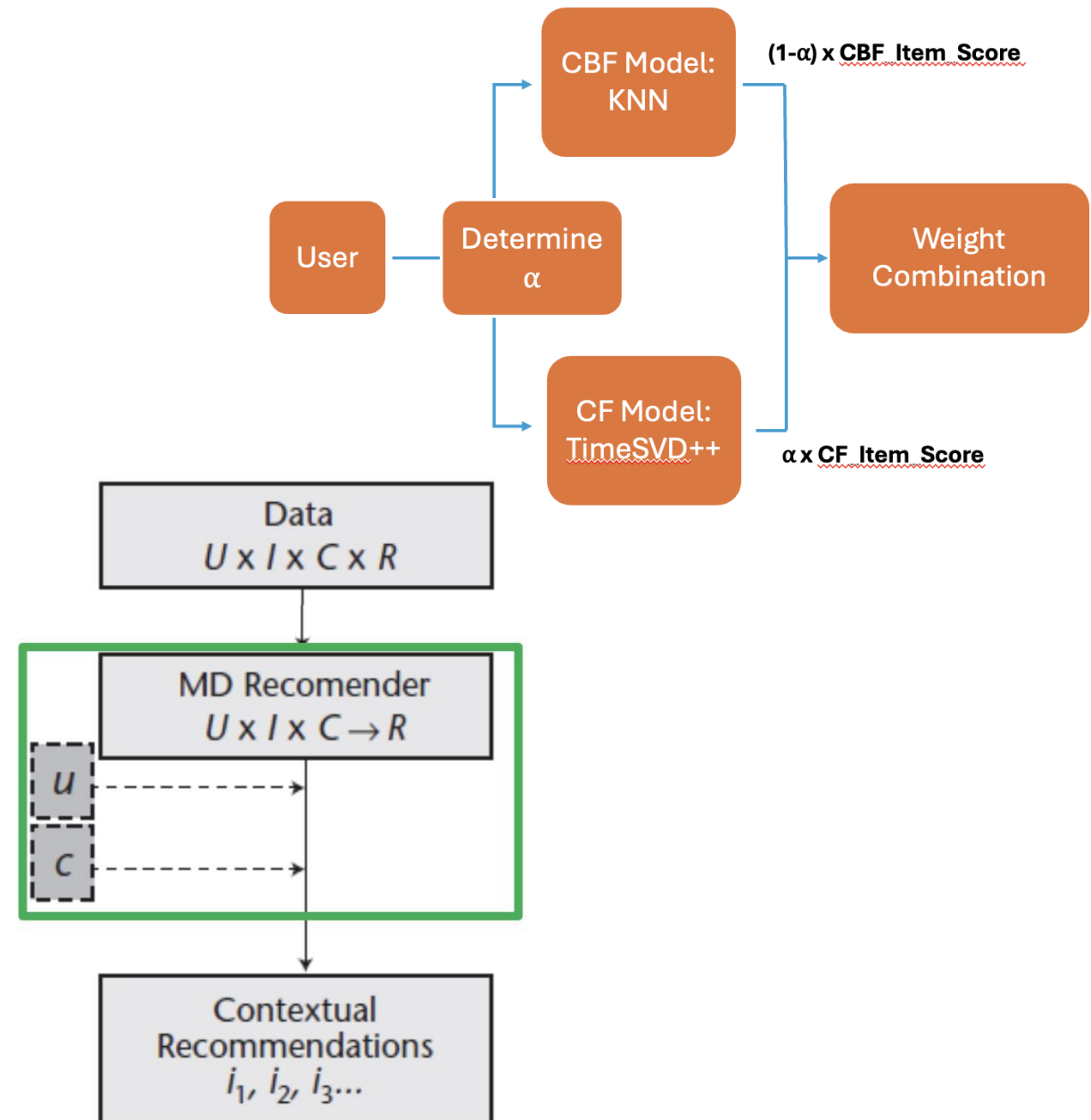
- Uses **TimeSVD++** to model time-dependent user bias, item bias, and preference drift
- Interaction timestamps are discretised into coarse time bins.
- **Time treated as an explicit, interpretable context feature**

RS2: Implicit Temporal Context

- Uses **SASRec** to model the **sequence and recency** of user interactions
- Learns temporal relevance directly from ordered behaviour
- **Time captured implicitly without manual discretisation**

Why:

- User preferences and item popularity are **non-stationary**
- Short-video consumption exhibits strong **recency and trend effects**
- Temporal modelling enables recommendations to adapt as interests evolve



Evaluation Metric: Ranking Quality (NDCG@K)

(Normalized Discounted Cumulative Gain at K)
measures:

- How well a recommender ranks relevant items near the top of the recommendation list, up to position K.

Why NDCG@K:

- Rewards placing the most relevant items near the top of the feed
- Supports graded relevance (e.g. watch ratio buckets or weighted implicit events).
- Relevance is defined using implicit feedback from user interactions.
- Use **NDCG@10** and **NDCG@50** (top of feed + slightly deeper scroll).

$$NDCG@k = \frac{DCG@k}{IDCG@k}$$

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

$$IDCG@k = \sum_{i=1}^k \frac{2^{rel_{\sigma(i)}} - 1}{\log_2(i+1)}$$

Evaluation Method: Serendipity

Serendipity@K Offline Definition:

- How surprising recommendations are
- **Offline proxy for user surprise** (true surprise cannot be measured without user studies)

Why Serendipity:

- Accuracy metrics reward recommending **what the user already likes**, but do not capture **discovery**.
- Users consume content at high volume, making **unexpected recommendations** important for sustained engagement.
- Important to expose users to content **outside their usual niche** can increase long-term satisfaction and reduce filter-bubble effects.

How we calculate offline Serendipity:

- Treat serendipity as **relevant but unexpected recommendations**
- Unexpectedness is approximated as **inverse item popularity** (computed from training data)
- Only **relevant items** in the top-K list contribute to the score
- Average unexpectedness over K to normalise across users

$$\text{popularity}(i) = \frac{\text{Interactions of Item } i}{\text{Total Users}}$$

$$\text{unexpectedness}(i) = 1 - \text{popularity}(i)$$

$$\text{Serendipity@K} = \frac{1}{K} \sum_{i \in \text{Top-K}} \mathbb{1}[\text{relevant}(i)] \cdot (1 - \text{popularity}(i))$$

$\mathbb{1}[\text{relevant}(i)]$ means item appears in test positives

```
# NDCG
```

```
dcg = np.sum([rel / np.log2(idx + 2) for idx, rel in enumerate(hits)])
ideal_hits = sorted(hits, reverse=True)
idcg = np.sum([rel / np.log2(idx + 2) for idx, rel in enumerate(ideal_hits)])
metrics[k]['ndcg'].append((dcg / idcg) if idcg > 0 else 0.0)
```

```
# Serendipity
```

```
ser_val = sum(get_unexpectedness(item) for item in top_k if item in true_positives)
metrics[k]['serendipity'].append(ser_val / k)
```

```
# RS1 Hybrid
```

```
def rs1_hybrid_scorer(user_id, items):
    c_scores = normalize_scores(knn_scorer(knn_rs1, user_id, items, train_history_rs1))
    l_scores = normalize_scores(timesvd_scorer(user_id, items))
    alpha = get_alpha(user_id)
    return {vid: alpha * l_scores.get(vid, 0) + (1 - alpha) * c_scores.get(vid, 0) for vid in items}

results['RS1_TimeSVD++'] = evaluate_scorer("TimeSVD++", timesvd_scorer)
results['RS1_KNN'] = evaluate_scorer("KNN", lambda u, i: knn_scorer(knn_rs1, u, i, train_history_rs1))
results['RS1_Hybrid'] = evaluate_scorer("Hybrid RS1", rs1_hybrid_scorer)
print("RS1 evaluated")
```

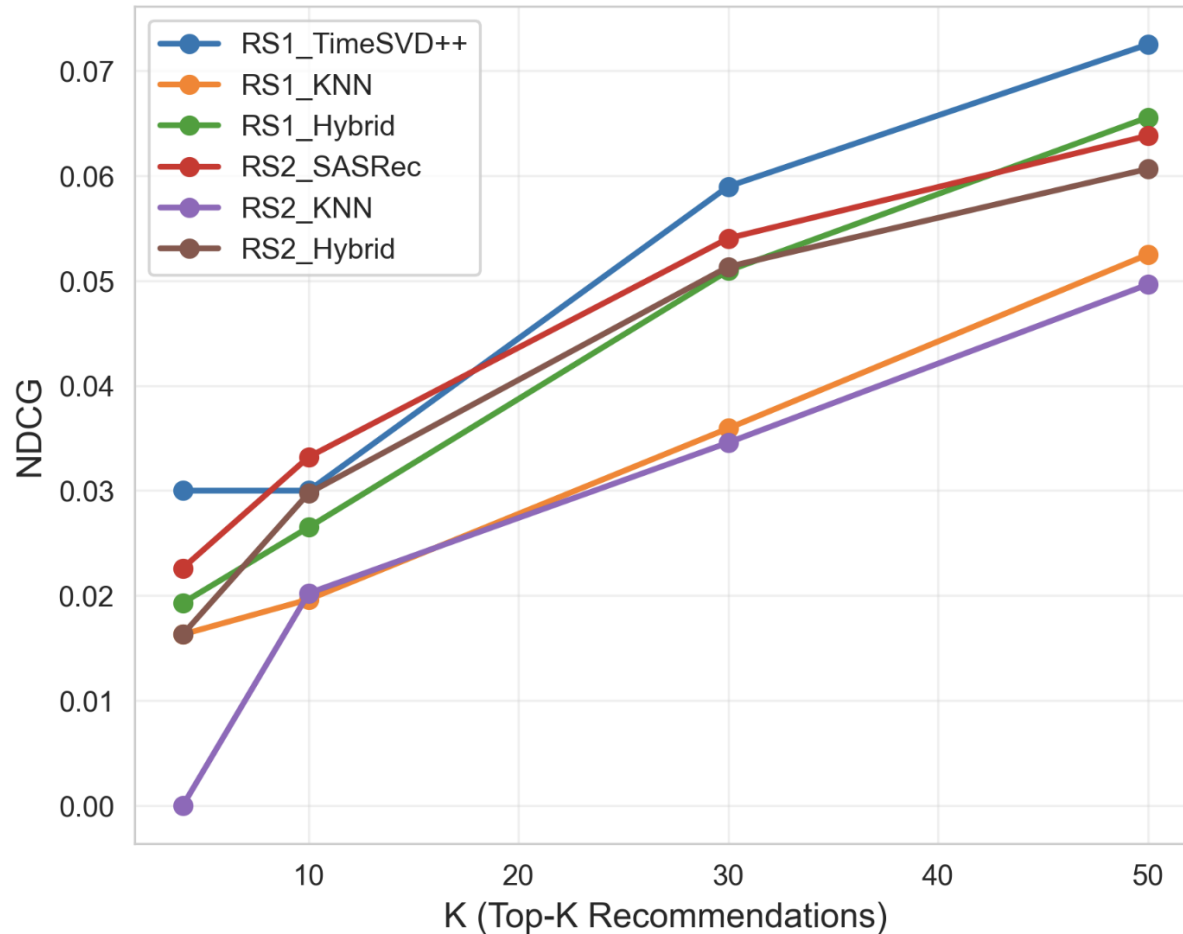
```
# RS2 Hybrid (SASRec + KNN)
```

```
def rs2_hybrid_scorer(user_id, items):
    s_scores = normalize_scores(sasrec_scorer(user_id, items))
    c_scores = normalize_scores(rs2_knn_scorer(user_id, items))
    alpha = get_alpha_rs2(user_id)
    return {vid: alpha * s_scores.get(vid, 0) + (1 - alpha) * c_scores.get(vid, 0) for vid in items}

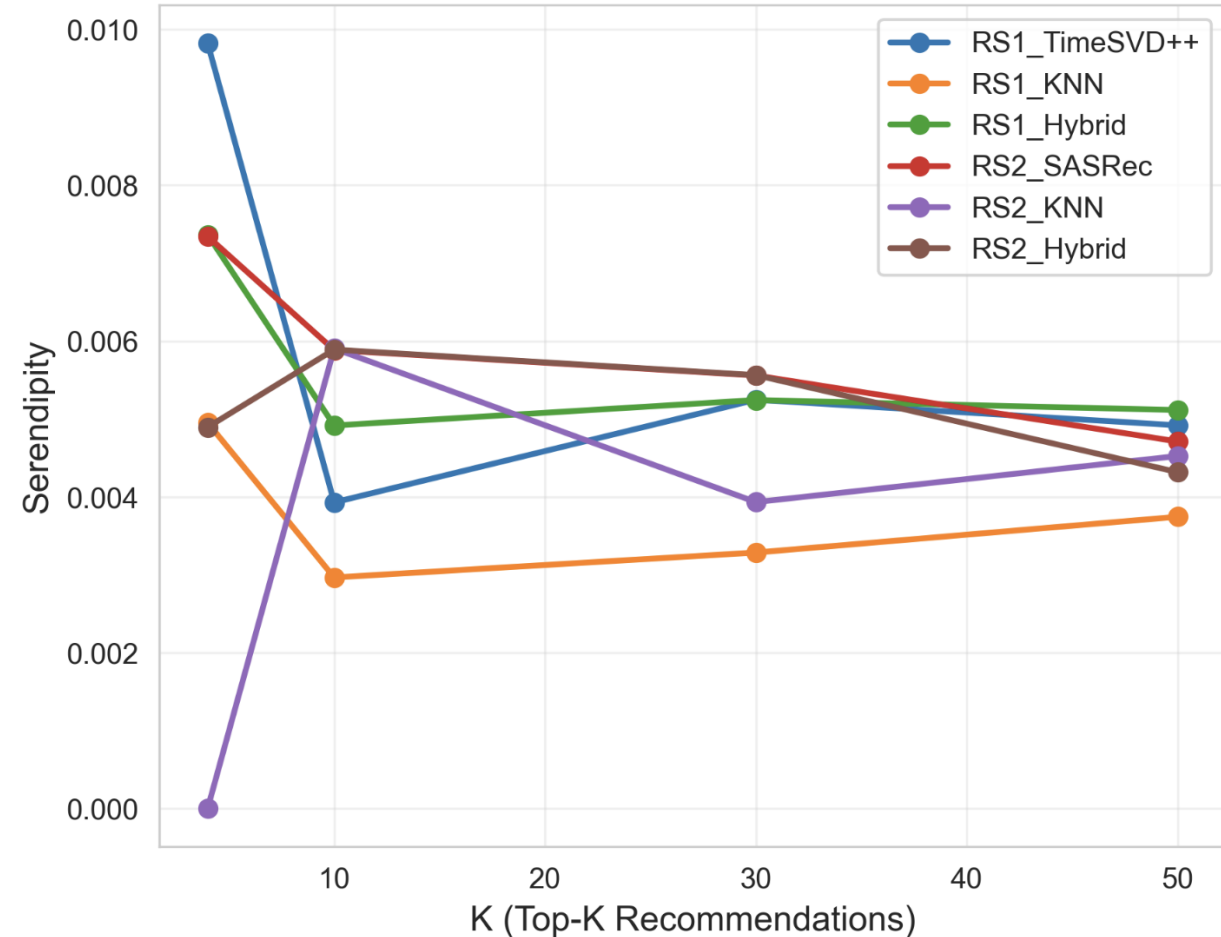
results['RS2_SASRec'] = evaluate_scorer("SASRec", sasrec_scorer)
results['RS2_KNN'] = evaluate_scorer("KNN", rs2_knn_scorer)
results['RS2_Hybrid'] = evaluate_scorer("Hybrid RS2", rs2_hybrid_scorer)
print("    ✅ RS2 evaluated with SASRec model")
```

Comparing RS1 and RS2

NDCG Comparison



Serendipity Comparison





Responsible AI Consideration

Popularity Bias & Filter Bubbles:

- Both RS1 (TimeSVD++) and RS2 (SASRec) risk over-exposing popular content.
- This can **reduce content diversity and reinforce existing user preferences**.
- Mitigation:
 - Use of **Serendipity@K** to explicitly evaluate exposure to less popular content.

User Autonomy & Manipulation Risk:

- **Sequential models (RS2) can strongly optimise for short-term engagement.**
- Risk of steering users toward **addictive or narrow content loops**.
- Mitigation:
 - Offline evaluation only (no real-time optimisation).
 - Evaluation focuses on ranking quality, not maximising watch time.

Fairness Across Content Creators:

- Popularity-driven recommendation can disadvantage niche or new creators.
- **RS2's higher capacity may amplify these effects.**
- Mitigation:
 - Serendipity metric encourages exposure to less-frequent items.

Display Interface Showcase!



References

1. C. Gao, S. Li, W. Lei, J. Chen, B. Li, P. Jiang, X. He, J. Mao and T.-S. Chua, “KuaiRec: A Fully-Observed Dataset and Insights for Evaluating Recommender Systems,” in *Proc. 31st ACM Int. Conf. on Information and Knowledge Management (CIKM '22)*, Atlanta, GA, USA, Oct. 17-21 2022, pp. xxx-xxx, ACM, 2022, doi:10.1145/3511808.3557220.
2. B. Sarwar, G. Karypis, J. A. Konstan and J. Riedl, “Item-Based Collaborative Filtering Recommendation Algorithms,” in *Proc. 10th Int. Conf. on World Wide Web (WWW '01)*, Hong Kong, May 1-5 2001, pp. 285-295, ACM, 2001, doi:10.1145/371920.372071.
3. Desrosiers, C., & Karypis, G. (2011). A comprehensive survey of neighborhood-based recommendation methods. In *Recommender systems handbook* (pp. 107-144). Springer, Boston, MA.
4. Koren, Y., & Bell, R. (2011). Advances in collaborative filtering. *Recommender systems handbook*, F. Ricci, L. Rokach, B. Shapira, and PB Kantor.
5. Schafer, J. B., Frankowski, D., Herlocker, J., & Sen, S. (2007). Collaborative filtering recommender systems. In *The adaptive web* (pp. 291-324). Springer, Berlin, Heidelberg.
6. Lops, P., De Gemmis, M., & Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In *Recommender systems handbook* (pp. 73-105). Springer, Boston, MA.
7. Wang, D., Liang, Y., Xu, D., Feng, X., & Guan, R. (2018). A content-based recommender system for computer science publications. *Knowledge-Based Systems*, 157, 1-9.
8. Amatriain, X., Jaimes, A., Oliver, N., & Pujol, J. M. (2011). Data mining methods for recommender systems. In *Recommender systems handbook* (pp. 39-71). Springer, Boston, MA.
9. J. Kim, J. Wi, and Y. Kim, “Sequential Recommendations on GitHub Repository,” *Applied Sciences*, vol. 11, no. 4, Art. no. 1585, Feb. 2021, doi: 10.3390/app11041585.
10. Adomavicius, G., & Tuzhilin, A. (2011). Context-aware recommender systems. In *Recommender systems handbook* (pp. 217-253). Springer, Boston, MA.
11. W.-C. Kang and J. McAuley, “Self-Attentive Sequential Recommendation,” in *Proc. IEEE Int. Conf. on Data Mining (ICDM)*, Singapore, 2018, pp. 197–206.
12. K. Järvelin and J. Kekäläinen, “Cumulated Gain-Based Evaluation of Information Retrieval Techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, Oct. 2002.
13. T. Murakami, K. Mori, and R. Orihara, “Metrics for Evaluating the Serendipity of Recommendation Lists,” in *Proc. ACM Int. Conf. on Intelligent User Interfaces (IUI)*, Honolulu, HI, USA, 2007, pp. 40–46.
14. S. Vargas and P. Castells, “**Rank and Relevance in Novelty and Diversity Metrics for Recommender Systems**,” in *Proc. ACM Conf. on Recommender Systems (RecSys)*, Chicago, IL, USA, 2011, pp. 109–116.
15. R. Burke, A. Abdollahpouri, M. C. Jones, A. Malthouse, and K. G. Strokova, “**Multisided Fairness for Recommendation**,” in *Proc. ACM Conf. on Fairness, Accountability, and Transparency (FAT*)*, New York, NY, USA, 2018, pp. 1–10.
16. H. Miller, D. Kluver, J. Thebault-Spieker, L. Terveen, and B. Hecht, “Understanding Emoji Ambiguity in Context: The Role of Text in Emoji-Related Miscommunication,” in *Proc. 11th Int. AAAI Conf. Web and Social Media (ICWSM)*, Palo Alto, CA, USA, 2017, pp. 152-161.